

FinOps Cost Analysis Report

AWS 인프라 비용 최적화 분석 리포트

생성일: 2026년 05월 26일

3

발견된 이슈

\$1,632.00

월간 절감 예상액

85%

분석 신뢰도

발견된 비용 낭비 이슈

Lambda 함수들이 메모리를 과도하게 할당받아 비용 낭비 발생

리소스: comp1_lambda-function-xoq9qq, comp1_lambda-function-0n5puy, comp1_lambda-function-j5dyhj | 유형: overprovisioned

\$750.00/월

1. Problem Identification

3008MB로 설정된 메모리 대비 실제 사용량이 평균 100MB로 30배 과잉 할당

2. Root Cause

실제 메모리 사용량 분석 없이 과도한 메모리 할당

- 실제 메모리 사용량: mean=99.98MB, p95=100.0MB, max=100.0MB
- 할당된 메모리: 3008MB (실사용량의 30배 과잉)

3. Proposed Solution

메모리 할당량을 실사용량 기준으로 최적화

- 메모리 할당량을 3008MB에서 512MB로 조정
- CloudWatch Lambda Insights로 지속적인 메모리 사용량 모니터링

4. Estimated Monthly Savings

\$750.00

월간 예상 절감액 (USD)

5. Savings Calculation Basis

메모리 할당량 감소로 GB-초 단가 절감

계산식: 3 함수 × \$250/month = \$750.00/month

- Lambda 요금: Duration × 할당 메모리 GB-초
- 3008MB → 512MB 조정 시 함수당 \$250/월 절감

Lambda 함수들의 타임아웃이 과도하게 설정되어 에러 시 비용 증가

리소스: comp2_lambda-function-w8lsz8, comp2_lambda-function-bybdc5 |
유형: overprovisioned

\$360.00/월

1. Problem Identification

900초 타임아웃 설정으로 에러 발생 시 최대 900초까지 과금되어 비용 낭비

2. Root Cause

실제 실행 시간 대비 과도한 타임아웃 설정과 에러율 존재

• 평균 실행 시간: 2141ms, 2124ms (약 2초)
• 에러율: 1.95%, 1.98% 지속 발생
• 타임아웃 900초로 에러 시 450배 과잉 과금

3. Proposed Solution

타임아웃을 실행 시간 기준으로 최적화

- 타임아웃을 900초에서 10초로 조정
- 에러 발생 원인 분석 및 알람 설정

4. Estimated Monthly Savings

\$360.00	월간 예상 절감액 (USD)
----------	-----------------

5. Savings Calculation Basis

타임아웃 단축으로 에러 시 과금 시간 99% 절감

계산식: 2 함수 × \$180/month = \$360.00/month

- 에러 시 900초 → 10초 조정으로 99% 절감
- 함수당 \$180/월 절감

DynamoDB 테이블의 프로비저닝된 용량이 실사용량 대비 과도하게 설정

리소스: comp3_dynamodb-table-bh7jai | 유형: overprovisioned

\$522.00/월

1. Problem Identification

RCU 5000, WCU 1000으로 프로비저닝했으나 실제 사용량은 훨씬 낮아 비용 낭비

2. Root Cause

Auto Scaling 없이 고정 프로비저닝으로 피크 기준 과잉 할당

- WCU 사용률: mean=85.65, 최소 13.07까지 하락
- RCU 사용률: mean=100.0으로 지속적 사용
- Auto Scaling 설정 없어 트래픽 변동에 대응 불가

3. Proposed Solution

On-Demand 모드 전환 또는 Auto Scaling 적용

- PROVISIONED에서 PAY_PER_REQUEST 모드로 전환
- 또는 aws_appautoscaling_target으로 Auto Scaling 설정

4. Estimated Monthly Savings

\$522.00	월간 예상 절감액 (USD)
----------	-----------------

5. Savings Calculation Basis

미사용 프로비저닝 용량 제거로 고정 비용 절감

계산식: 미사용 WCU 900 × \$0.00065/hr × 730hr + 미사용 RCU 4500 × \$0.00013/hr × 730hr = \$522.00/month

- WCU 단가: \$0.00065/시간
- RCU 단가: \$0.00013/시간
- 실사용량 대비 초과 프로비저닝 제거

단위 경제성 및 비즈니스 영향

단위 경제성 데이터 없음

태깅 및 탄력성

Governance Recommendations

- Terraform default_tags 도입: provider 수준 default_tags를 추가해 공통 태그(env/service/owner/cost_center)를 기본 적용

권고사항 (Recommendations)

조치	대상 리소스	우선순위	위험도	예상 절감액
Lambda 메모리 최적화	comp1_lambda-function-xoq9qq, comp1_lambda-function-0n5puy, comp1_lambda-function-j5dyhj	높음	낮음	\$750.00
Lambda 타임아웃 최적화	comp2_lambda-function-w8lsz8, comp2_lambda-function-bybdc5	높음	보통	\$360.00
DynamoDB 용량 모드 최적화	comp3_dynamodb-table-bh7jai	보통	낮음	\$522.00

멀티 에이전트 분석

항목	값
감지 도메인	lambda, dynamodb, governance
도메인 이슈	7
Cross-domain query	2
Cross-service finding	1
Recall 추정	single 1/3 (33.3%), multi 3/3 (100.0%)
추가 LLM 호출	1
추정 토큰	2,113
Wall-clock	12.57s

Lambda의 과도한 timeout 설정(900초)이 DynamoDB provisioned capacity 낭비를 증폭시키는 결합 낭비가 발견되었습니다. Lambda 에러 시 최대 900초 과금과 함께 DynamoDB 접근 패턴이 왜곡되어 실제 필요량 대비 과도한 capacity가 할당되고 있습니다.

Comparison Summary

{ "single_agent": { "recall": 0.333, "input_tokens": 2435, "output_tokens": 1189, "wall_clock_sec": 29.68 }, "multi_agent": { "recall": 1.0, "input_tokens": 1779, "output_tokens": 334, "wall_clock_sec": 12.57, "agent_count": 3 }, "framework": "lightweight rule-based analyzer", "notes": "Single-agent는 필수 3개 패턴 중 1개를 감지했고, multi-agent는 3개를 감지함. Multi-agent는 cross-service finding 도출 목적." }

Cross-Domain Query Loop

lambda → dynamodb (answered): MA-001 유형의 Lambda-DynamoDB 결합 낭비 판단
dynamodb → lambda (answered): DynamoDB provisioned 유지와 on-demand 전환 중 어느 쪽이 적합한지 판단

[high] api_gateway_lambda_dynamodb_cascade_amplification

Scenario: API Gateway -> Lambda -> DynamoDB cascade amplification

Services: Lambda, DynamoDB

Chain: Lambda → DynamoDB

Lambda → DynamoDB: excessive timeout extends retry-driven compute cost and amplifies provisioned capacity waste

Evidence: lambda: comp2_lambda-function-w8lsz8과 comp2_lambda-function-bybdc5에서 timeout 900s 대비 P99 6303ms/4924ms로 에러 시 최대 900s 과금; dynamodb: comp3_dynamodb-table-bh7jai에서 provisioned capacity 대비 사용량이 낮고 throttling 신호가 약함

Lambda 에러 시 900초 동안 과금되면서 DynamoDB 접근 패턴이 불규칙해지고, 이로 인해 provisioned capacity가 실제 필요량보다 과도하게 할당됨

권고: Lambda timeout을 11초/10초로 축소하고 DLQ 설정 후, DynamoDB를 PAY_PER_REQUEST로 전환하여 burst 패턴에 맞는 과금 모델 적용